CUDAnet · COMPANION DOCUMENT 2 OF 2

The CUDAnet Paper, Explained

A section-by-section study guide to *"CUDAnet: A Micro-Footprint, Dependency-Free Deep Learning Engine for Edge-Native GPU Training"* — the claims, the evidence behind each one, the math spelled out, and a prepared-answers list for the toughest questions a reviewer or the audience might ask.

ICNPCV 2026 · Samartha H V, Chiranth R, Pavithra N

Companion to: "The CUDAnet Codebase, Explained"

Contents

1. 1 TL;DR
2. 2 Why This Paper Exists
3. 3 The Core Bug: Race Conditions in Stateful GPU Optimizers
4. 4 The Fix: Two-Phase Gradient Accumulation (2PGA)
5. 5 Architecture: MLP + Batch Normalization
6. 6 Optimizer Math Reference
7. 7 Experimental Setup, Recapped
8. 8 Results Deep-Dive, Table by Table
9. 9 The BN + Adam Mystery
10. 10 The Edge AI Case, Interpreted
11. 11 Limitations, Honestly Assessed
12. 12 Anticipated Q&A
13. 13 Glossary & Notation

1. TL;DR

You built a neural-network training engine entirely from scratch in raw CUDA C — no PyTorch, no cuBLAS, no cuDNN. While doing so, you found that **parallelizing Momentum or Adam naively across GPU threads corrupts their optimizer state** and silently collapses accuracy to random chance, even though the program compiles and runs without any error. You diagnosed the exact mechanism (a classic read-modify-write race on shared velocity/moment buffers), fixed it with a simple two-step pattern you call **Two-Phase Gradient Accumulation (2PGA)**, and then benchmarked six configurations on MNIST to show the fix works, the engine is fast (>2,000× a CPU baseline), and the resulting binary is small enough (<2 MB) to matter for edge deployment.

> **The one sentence you should be able to say without notes**
>
> "Momentum and Adam keep state across batches; when you parallelize their update across 2,048 GPU threads without synchronizing, concurrent writes to that state silently corrupt it, dropping accuracy from 97% to 0% — and the fix is to separate the parallel gradient computation from the sequential state update with a single barrier."

2. Why This Paper Exists

The related-work section positions the paper against three clusters of prior work, and it's worth knowing exactly what gap each one leaves:

Cluster	Representative prior work	What they did NOT address
GPU-accelerated NN speedups	Wang et al. (17×), Rao et al. (487× backprop), Ramroach et al. (65×), Zhao et al. (110×, 97.2% MNIST)	None explicitly identify or resolve race conditions in stateful GPU-parallel optimizers
Optimizer parallelism	Kim & Park (distributed SGD, multi-GPU), Dean et al. (async SGD, accepts stale gradients)	Multi-GPU communication correctness, or side-stepping the problem via staleness — neither addresses *within-GPU* state correctness synchronously
Memory / compute optimization	Li et al. (memory patterns), Huang et al. (Tensor-Core pruning), Xu et al. (mixed precision), Tan & Chen (hybrid CPU-GPU)	Orthogonal to correctness — these are compatible with 2PGA, not competitors to it

The positioning claim, precisely: **2PGA is a synchronous, zero-staleness alternative** to Dean et al.'s async approach — it doesn't accept stale gradients to sidestep the race, it eliminates the race while keeping every gradient fresh.

3. The Core Bug: Race Conditions in Stateful GPU Optimizers

Why does this bug exist at all? Because the forward and backward passes for independent samples *are* embarrassingly parallel — thread *i* processing sample *i* never needs to know what thread *j* is doing. The optimizer **update** step breaks that independence: Momentum keeps one velocity buffer v_t shared across *all* samples in the batch, and Adam keeps two, m_t and v_t. Every thread wants to touch the *same* memory.

The race, spelled out

$$v_t^{new} = \beta \cdot v_t + \nabla_W L^{(i)}$$

Thread *i* reads v_t, computes a new value using its own gradient $\nabla_W L^{(i)}$, and writes it back. If thread *j* reads the *same* v_t before thread *i*'s write lands, thread *j*'s subsequent write overwrites thread *i*'s update entirely — thread *i*'s contribution to the velocity buffer simply vanishes. With B = 2,048 threads hammering the same address every step, this isn't a rare unlucky interleaving — it is **deterministic and total**: essentially every update after the first gets clobbered.

Why it's so easy to miss

- The kernel compiles cleanly — there's no type error, no illegal memory access, nothing CUDA's error-checking will flag.
- It runs to completion — no crash, no NaN necessarily, just wrong numbers.
- SGD — the simplest, usually-first-implemented optimizer — has **no state** ($v_t = \nabla_W L$ directly, nothing carried between steps), so it keeps working perfectly the whole time. A developer who validates "the pipeline works" using SGD and only later switches on Momentum/Adam will see a mysterious accuracy collapse with zero compiler or runtime signal pointing at the cause.

The measured damage

Configuration	Accuracy without the fix	Accuracy with 2PGA
SGD (stateless)	74.9% — the gap here is from single-batch vs. full-dataset training, not the race, since SGD has no race to begin with	94.3%
Momentum	0.0%	97.2%
Adam	0.0%	97.2%
BN + Adam + LR	17.9%	81.0%

Note the BN+Adam+LR row isn't 0% — batch normalization's own re-centering of activations partially masks the corrupted optimizer state, producing a low-but-nonzero accuracy rather than pure chance. That's a coincidence of BN's math, not evidence the race was less severe there.

4. The Fix: Two-Phase Gradient Accumulation (2PGA)

The fix follows directly from naming the actual conflict precisely: it's not the *gradient computation* that's unsafe in parallel (that genuinely is embarrassingly parallel, one sample per thread, no cross-thread dependency) — it's the *optimizer state update* that needs exactly one writer per parameter. So split the batch computation into two serialized phases with a hard barrier between them.

Phase 1 — Parallel Gradient Accumulation

$$G_W = \Sigma_{i=1..B} \nabla_W L_i \quad (via\ atomicAdd)$$

Each of the B threads independently computes its full forward pass, backward pass, and per-sample gradient, then adds its contribution into a shared accumulation buffer using `atomicAdd()`. This is safe by a different mechanism than Phase 2: `atomicAdd` serializes the individual additions *at the hardware level*, so no contribution is lost — unlike the naive read-modify-write race, addition is commutative and the hardware guarantees each add is atomic. Critically, **no optimizer state (velocity, moments) is touched here at all** — only the raw gradient sum.

The barrier

`cudaDeviceSynchronize()` — every thread across every block must finish Phase 1 before any thread starts Phase 2. This is what makes the two phases genuinely sequential rather than just "hopefully mostly done."

Phase 2 — Sequential State Update

$$W_{t+1} = W_t - \eta \cdot optimizer(G_W / B)$$

A separate kernel launch reads the now-complete G_W once and applies the optimizer update exactly once per parameter. The key structural property: **thread *k* is assigned to parameter *k*, and only parameter *k*** — there is no scenario where two threads in Phase 2 touch the same memory address, so there is nothing for a race condition to happen *to*. This is why the paper can say Phase 2 needs no atomics and no locking: not because they were optimized away, but because the one-thread-per-parameter assignment makes concurrent writes to the same address structurally impossible.

Why the cost is negligible

For the paper's 128×128 model, Phase 2 touches ~18,000 weights and finishes in <0.01 ms — under 0.4% of a single epoch's time. The argument scales: assign one thread to exactly one weight across however many thread blocks the parameter count requires, and the "one owner per address" property holds regardless of model size — the paper argues this reasoning carries to Transformer-scale parameter counts without modification.

2PGA is not "mini-batch SGD with extra steps"

Worth being precise about this if asked: Phase 1 produces exactly the same batch gradient G_W that ordinary gradient accumulation would — that part of the computation is unchanged. What 2PGA adds is entirely in *how the state update is applied*: by enforcing a global barrier and giving each parameter exactly one writer, it guarantees the state update happens exactly once per batch, with no interleaving. The contribution is a correctness guarantee on the optimizer's internal state, not a new gradient-accumulation algorithm.

5. Architecture: MLP + Batch Normalization

Default MNIST configuration: input $\mathbf{x} \in \mathbb{R}^{784}$ (flattened 28×28), n_l hidden layers of n_h ReLU neurons each, softmax output over 10 classes. Cross-entropy loss:

$$\mathscr{L} = -(1/N)\ \Sigma_{i=1..N}\ log\ \hat{y}_{i,c_i}$$

Batch Normalization, and why it needs its own barrier

For hidden layer l with pre-activation $z^{(l)}$, BN normalizes each feature j over the mini-batch B:

$$\mu_j = (1/|B|)\Sigma_i\ z_{ij}^{(l)} \qquad \sigma_j^2 = (1/|B|)\Sigma_i(z_{ij}^{(l)} - \mu_j)^2$$

$$\hat{z}_{ij} = (z_{ij}^{(l)} - \mu_j)\ /\ \sqrt{(\sigma_j^2 + \varepsilon)} \qquad \zeta_{ij} = \gamma_j\hat{z}_{ij} + \beta_j$$

Computing μ_j and σ_j^2 requires **every sample's z-value already written** before any thread reads them to compute the mean. A single unsynchronized kernel launch cannot guarantee this — threads in different blocks run concurrently with no ordering guarantee between blocks. This is a *different* race than the optimizer one in §3 (it's a read-before-write hazard on activations, not a write-write hazard on state), but the fix has the same shape: separate the phases with a barrier. The forward pass becomes a staged sequence — linear transform, `cudaDeviceSynchronize()`, BN statistics, BN apply, activation — repeated per layer. The backward pass mirrors this with its own synchronization point.

6. Optimizer Math Reference

Optimizer	Update rule
SGD	$W_{t+1} = W_t - \eta \nabla_W \mathscr{L}$
Momentum (β=0.9)	$v_t = \beta v_{t-1} + (1-\beta)\nabla_W \mathscr{L}_t \quad W_{t+1} = W_t - \eta v_t$
Adam (β_1=0.9, β_2=0.999, ε=10^{-8})	$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$ $\hat{m}_t = m_t/(1-\beta_1^t) \quad \hat{v}_t = v_t/(1-\beta_2^t) \quad W_{t+1} = W_t - \eta \hat{m}_t/(\sqrt{\hat{v}_t} + \varepsilon)$

Learning-rate scheduling: the step-vs-epoch bug that mattered more than expected

Cosine annealing:

$$\eta_s = (\eta_0/2)(1 + \cos(\pi s/S))$$

where S = total step count and s = current global step. The paper flags a specific implementation detail as more consequential than expected: whether s indexes by **gradient step** or by **epoch**. Indexing by epoch compresses the entire cosine decay into just 23 steps (one per epoch instead of one per mini-batch, since there are 23 mini-batches/epoch) — the learning rate collapses to near-zero long before the network has converged. Indexing by the true global step (epoch × batches_per_epoch + batch) fixes this. This single indexing choice is what separates a working schedule from a broken one, not the annealing formula itself.

7. Experimental Setup, Recapped

Axis	Value
Hardware	NVIDIA RTX A2000, 12GB, sm_86 (Ampere), 3,328 CUDA cores, 448 GB/s bandwidth
Software	`nvcc -arch sm_86 -O2`, g++-12, driver 535.288.01
Dataset	MNIST, 60k train / 10k test, 28×28 grayscale; 80/20 train/val split → 48k / 12k
Model	2 hidden layers × 128 neurons, Xavier init ($\sigma=\sqrt{(2/n_{in})}$), biases = 0
Training	20 epochs, batch 2,048 → $\lfloor 48000/2048 \rfloor = 23$ mini-batches/epoch
Learning rates	0.1 for SGD/Momentum, 0.01 for Adam/Adam+LR
Metrics	Test accuracy (primary), train accuracy, time/epoch (CUDA events), NVML power sampling, CPU→GPU speedup ratio

8. Results Deep-Dive, Table by Table

Table I — Training Performance

The headline number: CUDA SGD completes an epoch in 0.94 s versus an estimated 2,070 s for serial CPU (90 s/batch × 23 batches) — a **2,202×** speedup. Even the heaviest configuration, BN+Adam+LR at 1.24 s/epoch, still delivers **1,669×**. The 22% extra cost of BN+Adam+LR over plain Adam (1.24s vs 1.02s) is directly attributable to the additional per-layer BN statistics kernels and the `cudaDeviceSynchronize()` barriers between them — i.e. the cost of correctness for BN, not inefficiency.

Table II — Before/After the 2PGA Fix

This is the table that proves the paper's central claim. Read it as: SGD's improvement (74.9%→94.3%) is a *different effect* (full-dataset vs. single-batch training) layered on top of a race that never applied to it. Momentum and Adam's improvement (0.0%→97.2% each) is *purely* the race-condition fix — same architecture, same data regime, same hyperparameters, the only variable is whether Phase 2 exists. BN+Adam+LR's partial recovery (17.9%→81.0%) still leaves a real, separate problem uninvestigated at this point in the paper — that's §VII-F / §9 of this guide.

Table III — Optimizer Comparison

Momentum and Adam tie at 97.2%, both +2.9pp over SGD, and **neither functions at all without 2PGA** — that clause is the paper's way of forcing the reader to price in correctness before comparing accuracy numbers. Adam+LR gains essentially nothing over plain Adam at 20 epochs (97.0% vs 97.2%) because with only 23 steps/epoch × 20 epochs = 460 total steps, the cosine schedule barely has room to matter.

Table IV — Convergence Over Epochs

By epoch 5: Adam/Adam+LR already at 96.3% (0.9pp from final), SGD at 90.7% (3.6pp from final). Adam's adaptive per-parameter learning rates give it a better-conditioned gradient signal from the very first mini-batch. Momentum tracks Adam closely (95.0% at epoch 5). BN+Adam+LR is the outlier — it's *highest* at epoch 2 (88.1%) and monotonically erodes for the remaining 18 epochs down to 81.0%. That specific shape — peak early, decay steadily — is diagnostic: it reads as optimization **instability**, not slow convergence. A slow-converging model would still be climbing at epoch 20; this one peaked and reversed.

Table V — Batch Size Scaling

512→2048: utilization rises 76%→94% as kernel-launch overhead gets amortized over more work per launch. 2048→4096: epoch time rises 17% because Phase 2's synchronization overhead — fixed per batch, not per sample — starts to matter relative to a batch that's not proportionally larger in useful work. 2,048 is presented as the practical sweet spot for this GPU, not a universal constant.

Table VI — CUDAnet vs. PyTorch

Covered in depth in §10 below.

9. The BN + Adam Mystery

This is the paper's most interesting open problem, and the section most likely to draw follow-up questions, so it's worth having the full chain of reasoning ready.

What was observed

- BN+Adam+LR peaks at epoch 2 (88.1%) then erodes steadily to 81.0% by epoch 20 — a 7.1pp regression *after* the model already reached a better state.
- BN+SGD, by contrast, loses almost nothing to BN: 93.1% vs. 94.3% plain SGD (−1.2pp), and does *not* show the peak-then-decay pattern.

Hypotheses tested and ruled out

> **Hypothesis: the LR schedule is over-decaying**
>
> Tested by switching from per-epoch to per-batch cosine indexing (the fix described in §6). Result: only **+1.1pp** recovered (79.9%→81.0%). If the schedule were the main cause, fixing its most severe implementation bug should have recovered far more than 1.1 percentage points. **Ruled out as the primary cause** — though it is a real, independently-worthwhile fix.

Current best explanation

Adam and BN both maintain *adaptive, history-dependent statistics* that are computed from the same evolving activations — Adam's m_t/v_t moment estimates are computed from gradients that flow through BN's per-batch mean/variance, which themselves shift every batch as the weights change. The paper frames these as two **"competing moving targets"**: Adam is trying to build a stable estimate of the gradient's second moment at the same time BN is repeatedly re-normalizing the very activations that gradient depends on. SGD has no such competing adaptive state, which is consistent with why BN+SGD is stable while BN+Adam+LR is not.

Proposed remedy (future work, not yet implemented)

Decoupled learning rates for BN parameters (γ, β) versus the main weights — i.e. don't let Adam's adaptive step size for the weights be driven by gradients that are entangled with BN's own fast-moving statistics. Alternatively, a BN-aware Adam variant.

> **If asked "why didn't you just fix it?"**
>
> Honest answer: the paper explicitly frames 81.0% as "genuinely unsatisfactory" and states the epoch-2 peak/decay pattern is *reproducible across runs* — this isn't a one-off unlucky seed, it's a structural interaction that needs a targeted architectural change (decoupled BN learning rates), which is scoped as future work rather than attempted within this paper's scope.

10. The Edge AI Case, Interpreted

Metric	CUDAnet	PyTorch	Reading
Test accuracy (20 ep.)	97.0%	97.7%	PyTorch wins slightly — cuBLAS/cuDNN kernels are more numerically refined
Time/epoch	~0.95s	0.25s	PyTorch is ~4× faster per epoch — expected, given years of kernel-fusion engineering
Peak VRAM	~470KB	248MB	**527× smaller**
Install size	<2MB	1.8GB	**900× smaller**
Runtime deps	`libcuda.so` only	Python + libtorch	No interpreter, no package manager needed on target
Startup time	<100ms	~3–5s	Matters for cold-start embedded inference

The framing to hold onto: this is **not** a claim that CUDAnet beats PyTorch on a workstation GPU — it plainly doesn't, on speed or accuracy. It's a claim that on a device with a ~2GB *total* RAM budget (a Jetson Nano, for instance), PyTorch's 1.8GB install and 248MB runtime footprint may not fit at all, while CUDAnet's <2MB / ~470KB footprint comfortably does. The Jetson Nano inference figure (~0.4ms/image, ~2,500 images/sec) clears the 30 FPS real-time threshold, which is the concrete bar for "usable for live embedded vision."

11. Limitations, Honestly Assessed

The paper states these itself — know them, because a reviewer/audience member who has read the paper will expect you to volunteer them, not need to be caught out by them:

- **MNIST MLP accuracy (97.2%) trails CNN results (>99%)** — by design: the MLP architecture isolates the parallel-correctness question from convolutional complexity, not because CNNs weren't considered.
- **BN+Adam+LR at 81.0% is a genuine unresolved weakness**, reproducible across runs, not fixed by the LR-schedule correction (§9).
- **Above batch size 4096, Phase 2 sync overhead becomes measurable** (~17% more epoch time at 4096 vs. 2048) — multi-stream pipelining is proposed as a mitigation, not yet implemented.
- **The serial CPU baseline (2,070 s/epoch) is an estimate**, not a completed full-dataset run — extrapolated from a measured 90s/batch × 23 batches. The paper is explicit that the 2,202× figure should be read as an order-of-magnitude result, not a precise measurement.

12. Anticipated Q&A

Ten questions most likely to come up in the 2-minute Q&A window, with a tight prepared answer for each.

Q: Why didn't you just use PyTorch/TensorFlow to begin with?

Because the paper's contribution is about correctness at the CUDA-kernel level — a race condition that exists whenever you hand-roll GPU-parallel stateful optimizers. Using a framework that already handles this correctly (via cuBLAS/cuDNN's own synchronization) would hide the exact problem the paper is about. The from-scratch build is also what makes the <2MB / zero-dependency edge-deployment story possible.

Q: Isn't this race condition just "well-known" / obvious to anyone who's done parallel programming?

The mechanism (read-modify-write race) is textbook parallel programming. What's not obvious is *how silently* it manifests in this specific context: no compiler error, no crash, and the most commonly-implemented-first optimizer (SGD) is immune, so a natural development workflow can validate "it works" before ever encountering the bug. The paper's contribution is diagnosing this specific failure mode in this specific domain and providing a named, reusable pattern (2PGA) — not claiming to have discovered races in general.

Q: Does 2PGA add meaningful overhead?

No — Phase 2 for the 128×128 model touches ~18,000 parameters and finishes in under 0.01ms, under 0.4% of an epoch. The extra cost visible in Table I (e.g., BN+Adam+LR's 1.24s vs. plain Adam's 1.02s) comes from BN's own synchronization needs, not from 2PGA itself.

Q: Would this scale to a Transformer / much larger model?

The paper argues yes, structurally: assign one thread to exactly one parameter in Phase 2, and the "single owner per address" property that makes the race impossible holds regardless of parameter count. This hasn't been empirically tested at Transformer scale in this paper — it's an architectural argument, not a benchmark claim.

Q: Why is BN+Adam+LR still broken?

See §9 above — competing adaptive statistics between Adam's moment estimates and BN's batch statistics. Ruled out the LR schedule as primary cause (only +1.1pp from fixing it). Proposed remedy is decoupled BN learning rates, scoped as future work.

Q: How did you measure the CPU baseline — is 2,070s trustworthy?

It's an estimate: one mini-batch measured at 90s on CPU, multiplied by 23 batches/epoch. The full-dataset CPU run was not completed end-to-end (would take ~11.5 hours for 20 epochs). Present the >2,000× figure as order-of-magnitude, which the paper itself does explicitly.

Q: What exactly is Two-Phase Gradient Accumulation — is it your invention or a known pattern?

The general idea of separating parallel accumulation from a sequential apply step is not novel in distributed systems generally. What the paper contributes is identifying that this specific two-phase discipline is *necessary* (not just an optimization) for correctness in GPU-parallel stateful optimizers, naming the pattern, and demonstrating the specific failure mode it fixes with measured before/after numbers.

Q: Why MLP and not CNN?

Deliberate scope control — an MLP is architecturally simple enough that the only source of GPU-parallel complexity is the optimizer/BN synchronization question the paper is actually about. Convolutional layers via im2col+GEMM targeting CIFAR-10 is explicitly future work.

Q: What's the single most surprising number in the paper?

Momentum and Adam going from **exactly 0.0%** — not "low," not "degraded," literally random-chance 0% — to 97.2% with a one-kernel-launch fix. It's surprising precisely because it demonstrates total, silent failure rather than a graceful degradation.

Q: If someone re-runs your code, will they get exactly these numbers?

Same architecture/hyperparameters/seed should reproduce closely, but the paper is upfront that the CPU baseline is an estimate and BN+Adam+LR's exact decay trajectory, while reproducible in shape, may vary slightly run to run given it reflects a genuine training instability rather than a deterministic bug.

13. Glossary & Notation

Symbol / term	Meaning
2PGA	Two-Phase Gradient Accumulation — the paper's fix (§4)
v_t, m_t	Momentum velocity / Adam first moment — per-parameter state carried across batches
G_W	Batch-summed gradient, produced by Phase 1
μ_j, σ_j^2	Per-feature batch mean/variance computed by Batch Normalization
γ, β	BN's own learnable scale and shift parameters (separate from the main weights)
`atomicAdd`	Hardware-serialized addition — safe for commutative accumulation, not a general race-condition fix
`cudaDeviceSynchronize()`	Global barrier — blocks the host until every thread across every block has finished
sm_86	NVIDIA "compute capability" 8.6 — the Ampere architecture generation used (RTX A2000)
pp	Percentage points (as opposed to relative %) — used throughout the results tables